



P000340SE

EVALUATION REPORT #2: BEDROCK

Product Owner: Branko Minic, Adrian Zielonka
Supervisor: Maria Spichkova

Authors : **Cassandra Tedcy Britto, s3922104**
 Di Wang, s4070702
 Gabby Sanchez, s4039866
 Jordan Chiou, s4068959
 Sneha Thankam Oommen, s4070343

DOCUMENT CONTROL

Version #	Implemented By	Implementation Date	Reviewed By	Approval Date	Reason
1	Gabby Sanchez	22/09/2025			Initial Unformatted Report
2	Gabby Sanchez	17/10/2025			Refinement of Unformatted Report
3	Gabby Sanchez	26/10/2025			Working Draft of Formatted Report
4	Gabby Sanchez	27/10/2025			Initial Refinement of Draft based on Supervisor Feedback
5	Gabby Sanchez	02/11/2025			Final Version

TABLE OF CONTENTS

ABSTRACT.....	4
1 INTRODUCTION.....	4
2 EVALUATION PLATFORM AND MODELS.....	4
3 PHASE 1 EVALUATION	4
3.1 Objective	4
3.2 Methodology	5
Evaluation design.....	5
Data Collection	5
Prompt Engineering	6
Predefined Category Set	6
Data Analysis	7
3.3 Findings.....	8
Overall Accuracy Comparisons	8
Per-Category Accuracy Comparisons.....	9
Output Array-length Accuracy Comparisons	10
Runtime Performance Comparisons	10
Token Usage Comparisons	11
3.4 Phase 1 Conclusions.....	11
4 PHASE 2 EVALUATION	11
4.1 Objective	11
4.2 Methodology	11
Evaluation design.....	11
Prompt Variants and Design Adjustments.....	12
Updated Predefined Category Set.....	13
Evaluation Procedure	14
Data Analysis	14
Cost-Optimisation Considerations	14
4.3 Findings.....	15
Strict Evaluation - Overall Accuracy Comparisons.....	15
Lenient Evaluation - Overall Accuracy Comparisons	15
Runtime Performance Comparisons	17
Token Usage Comparisons	17
4.4 Phase 2 Conclusions.....	18
5 OVERALL CONCLUSIONS.....	18
6 FURTHER WORK.....	18
REFERENCES.....	19

ABSTRACT

We evaluated four foundational models on AWS Bedrock to identify and refine a classifier for ReceiptCAT, a receipt-item categorisation application. The evaluation compared Claude 3.7 Sonnet, Claude 4 Sonnet, Mixtral 8x7B Instruct, and Mistral 7B Instruct across 118 receipts and 389 expense items, assessing accuracy, consistency, hallucination rate, runtime, and token usage as a cost proxy. Based on these results, we refined category definitions and prompt design to improve accuracy and cost efficiency. Claude 3.7 achieved the best overall balance of accuracy, stability, and cost, and further refinements with updated categories and rules improved performance, while few-shot examples increased cost without clear gains. We recommend Claude 3.7 with updated predefined categories and no-shot prompting as the preferred model for ReceiptCAT.

1 INTRODUCTION

ReceiptCAT is a receipt-item categorisation application that automatically assigns each expense item to a predefined category for reporting and budgeting purposes. Accurate categorisation is essential to ensure meaningful financial insights and a reliable user experience.

To select a suitable classification model, we conducted a structured evaluation of several foundational models available through AWS Bedrock. The goal was to determine whether these models could reliably interpret receipt data already available in text form and classify items according to the application's predefined categories. We compared their accuracy, consistency, runtime performance, and cost efficiency using a shared dataset of real receipts.

Based on these findings, we refined the category definitions and prompt design to improve classification accuracy and interpretability and tested whether few-shot examples would provide additional benefit. This iterative approach ensured that the final configuration achieved both high accuracy and cost-effective performance before integration into the ReceiptCAT application.

2 EVALUATION PLATFORM AND MODELS

AWS Bedrock provides access to multiple foundational models through a unified `InvokeModel` API that accepts a prompt and returns a model-generated response [1], [2]. This platform allows developers to test different language models using a consistent interface and security framework, making it suitable for controlled evaluations. Each request includes a structured JSON payload that specifies the model identifier, prompt content, and configuration parameters.

We compared four models hosted on Bedrock: two proprietary models from Anthropic (Claude 3.7 Sonnet and Claude 4 Sonnet) and two open-weight models from Mistral (Mixtral 8x7B Instruct and Mistral 7B Instruct). All models were evaluated on the same receipt dataset using identical schema-first prompts to ensure fair comparison. Although the `InvokeModel` API is shared across providers, prompt formatting differs by family: Claude models separate system and user messages, whereas Mixtral and Mistral use a single combined text input [1], [3].

3 PHASE 1 EVALUATION

3.1 OBJECTIVE

The primary objective of this phase of the evaluation was to identify the most suitable foundational model available through AWS Bedrock for integration into ReceiptCAT. To achieve this, we conducted a controlled comparison of four models: Claude 3.7 Sonnet, Claude 4 Sonnet, Mixtral 8x7B Instruct, and Mistral 7B Instruct. Each model's performance was assessed using identical input data and consistent prompt structures, with

evaluation criteria covering classification accuracy, consistency, hallucination rate, runtime, and token usage as a proxy for cost.

In addition to model selection, we aimed to establish a baseline understanding of how different model families interpret receipt item text and adhere to schema-based output requirements. The results from this stage were used to identify strengths and limitations of each approach, providing a foundation for refinements later in the evaluation, during which category definitions and prompt design were systematically improved.

3.2 METHODOLOGY

Evaluation design

This evaluation followed a controlled comparison of four foundational models available through AWS Bedrock. Each model was treated as a black box, with identical inputs and parameters to ensure fair comparison. Only the prompt format varied between model families.

The process was automated using custom TypeScript scripts in the **bedrock-evaluation-scripts** subfolder of the **P000340SE-EvaluationReport-Appendix.zip** archive. This archive provides all relevant scripts, input datasets, and output files used throughout the evaluation. The main scripts are:

- **phase1/step1-model-evaluator.ts**, which invokes all models via the Bedrock `InvokeModel` API and logs predictions, prompts, tokens, and latency.
- **phase1/step2-ground-truth-evaluator.ts**, which compares model outputs against manually labelled ground-truth data to compute performance metrics.
- **phase1/prompt.ts**, which utilises the predefined category list and prompt templates.

Model invocations were executed by the model evaluator script (**phase1/step1-model-evaluator.ts**), which iterated over every receipt, invoked each model via the Bedrock `InvokeModel` API, and logged detailed outputs to CSV files. The script captured timing data, token counts (when reported), and validation errors for each call. Concurrency was kept minimal to avoid rate limiting and maintain consistent runtime conditions.

Correctness scoring and metric computation were performed by the ground truth evaluator script (**phase1/step2-ground-truth-evaluator.ts**), which compared each model's predictions with the manually integrated ground-truth column. All intermediate and final CSV outputs are included in the archive for reproducibility.

All evaluation runs were designed to minimise cost. Only the Bedrock `InvokeModel` API was used, with all processing performed locally to avoid charges from services such as S3, DynamoDB, Lambda, or Textract. Prompts were concise and free of few-shot examples to limit input tokens, and the required JSON-array output kept responses short. All input and output were handled as local CSV files, and successful results were cached to prevent unnecessary re-invocations. These measures ensured reproducibility while keeping per-receipt costs low.

Data Collection

The evaluation used a single CSV exported from the **AWS Textract Evaluation** as its primary data source. Each row represented one receipt, containing machine-extracted text blocks, the detected vendor, and the total amount. The dataset comprised **118 receipts** and **389 expense items**. This dataset is provided in the **P000340SE-EvaluationReport-Appendix.zip** archive under **data/textract_output_data.csv**. It served as input for all model evaluation scripts and was used consistently across both Phase 1 and Phase 2 to ensure comparable results.

Because the Textract input dataset was unevenly distributed, some categories, such as “*Pantry & Snacks*” and “*Eating Out*”, appeared more frequently than others, like “*Pets*” or “*Subscriptions & Digital Services*.” No receipts were available to represent “*Baby & Maternity*”, “*Entertainment*”, or “*Transport & Fuel*”, leaving those categories without ground-truth examples at this phase of the evaluation. Balanced accuracy and per-category reporting were therefore used to offset these imbalances. Ground truth labels were manually assigned to each receipt and item using the Phase 1 baseline category set. A short labelling guide clarified boundaries between similar categories, such as “*Eating Out*” versus “*Pantry & Snacks*” and “*Coffee & Tea*” versus “*Beverages*”. When items fit multiple plausible categories, we applied a strict interpretation and recorded only one target label.

Ground-truth labels were prepared manually by two team members (Gabby Sanchez and Sneha Thankam Oommen) of the ReceiptCAT development team. Each receipt item was inspected and assigned a single definitive category using the Phase 1 baseline category set. The resulting `ground_truth_category` column was then merged into the output from the Step 1 script before evaluation by the Step 2 evaluator. This process ensured that correctness scoring was based on verified human annotations.

Prompt Engineering

Prompts were designed to maintain identical task instructions and schema requirements across all model families while adapting to provider-specific input formats. For Claude models, a two-part structure was used: the `system` message defined the classification task, schema, and constraints, and the `user` message contained the serialised receipt text (vendor, total, and item list). For Mixtral and Mistral models, a single combined prompt embedded both the task instructions and the receipt data.

To ensure that the models remained tightly bound to valid outputs, each prompt explicitly included a predefined list of categories that are used by the ReceiptCAT application. By constraining the model to select from a fixed set rather than generating free-form labels, we reduced ambiguity and improved consistency across runs. This design choice also ensured alignment with the application’s reporting logic, which requires that each receipt item belong to exactly one recognised category.

This schema-first design improved parseability, minimised hallucinations, and reduced token usage through concise wording, following AWS Bedrock’s official guidance on concise, schema-driven prompt design [4]. Each prompt required a JSON array of category labels, each corresponding to the number of detected items, with no commentary or extra text. Simple heuristics, such as classifying cafés and restaurants as “*Eating Out*” or “*Coffee & Tea*”, were included to improve consistency.

The complete set of prompt templates, including schema definitions, heuristics, and model-specific variations, is provided in `phase1/prompt.ts` within the `P000340SE-EvaluationReport-Appendix.zip` archive.

Predefined Category Set

The predefined category set used in the prompts was created before any model evaluation. It was drafted collaboratively by the development team in consultation with the Product Owner, based on client meetings and early design discussions. To ensure relevance to Australian consumer spending, the team also reviewed how local financial and budgeting applications (such as the Commonwealth Bank mobile app) group transactions. The final list was designed to cover common household, personal, and service-related expenses while maintaining clear boundaries between similar categories.

The full list of 26 categories and their identifiers is shown in Table 4.2.1. The same list is hard-coded in the prompt templates defined within `phase1/prompt.ts` and `phase1/step1-model-evaluator.ts` within

the P000340SE-EvaluationReport-Appendix.zip archive, ensuring that all model runs referenced identical labels and schema definitions.

#	Category Name	Identifier
1	Fresh Produce	fresh_produce
2	Meat & Seafood	meat_seafood
3	Dairy & Eggs	dairy_eggs
4	Pantry & Snacks	pantry_snacks
5	Bakery	bakery
6	Coffee & Tea	coffee_tea
7	Beverages	beverages
8	Alcohol	alcohol
9	Eating Out	eating_out
10	Health & Medicine	health_medicine
11	Personal Care & Beauty	personal_care_beauty
12	Home & Cleaning	home_cleaning
13	Baby & Maternity	baby_maternity
14	Pets	pets
15	Clothing & Footwear	clothing_footwear
16	Electronics & Tech	electronics_tech
17	Stationery & Office	stationery_office
18	Sports & Fitness	sports_fitness
19	Gifts & Occasions	gifts_occasions
20	Entertainment	entertainment
21	Subscriptions & Digital Services	subscriptions_digital
22	Professional Services	professional_services
23	Utilities & Bills	utilities_bills
24	Transport & Fuel	transport_fuel
25	Travel & Holidays	travel_holidays
26	Other	other

Table 4.2.1 – Predefined Category Set used in Phase 1’s Model Evaluation

Data Analysis

Correctness was determined automatically by comparing each model’s predicted category with the manually assigned ground-truth label. A prediction was counted as correct only when it exactly matched the corresponding ground-truth category for that item.

Model performance was evaluated using several standard classification metrics:

- **Overall accuracy** represents the proportion of correctly predicted items across the entire dataset.
- **Balanced accuracy** is the average of per-category accuracies, giving equal weight to each category regardless of how many items it contains; this helps counter the imbalance in category frequencies.
- **Precision** measures the proportion of items a model correctly labelled as a given category.
- **Recall** measures the proportion of items from a given category that were successfully identified.
- The **F1 score** is the harmonic mean of precision and recall.

Metrics were calculated using both micro averaging (equal weighting of all predictions) and macro averaging (averaging scores per category). Together, these measures provide a balanced view of classification quality across frequent and infrequent categories.

All metrics were computed programmatically across all 389 items, along with per-category accuracy tables and support counts. Model outputs were first generated using the `phase1/step1-model-evaluator.ts` script, which produced two primary output files contained in the `data/bedrock_eval/phase1/step1_model_eval` subfolder within the `P000340SE-EvaluationReport-Appendix.zip` archive:

- **bedrock_output_phase1_eval_results.csv**, containing all per-item model predictions; and
- **bedrock_output_phase1_prompt_token_results.csv**, containing token usage and runtime data for each receipt.

Ground-truth categories were then manually added to the evaluation results file by two members (Gabby Sanchez and Sneha Thankam Oommen) of the ReceiptCAT development team, resulting in `bedrock_phase1_ground_truth_eval.csv`. This labelled file was subsequently used by the evaluator script (`phase1/step2-ground-truth-evaluator.ts`) to automatically compute all performance metrics.

Manual validation was limited to confirming that the added ground-truth labels were correctly aligned with the model outputs before automated scoring. All performance metrics were calculated using the complete dataset, ensuring accuracy, consistency, and reproducibility of the results.

3.3 FINDINGS

Overall Accuracy Comparisons

Model performance was evaluated by comparing each predicted category against the manually verified `ground_truth_category` column in the Phase 1 dataset. A prediction was counted as correct only when it exactly matched the corresponding ground-truth label for that item. Using these results, we calculated overall accuracy, balanced accuracy, precision, recall, and F1 scores for each model. The calculations were performed automatically using the evaluator script (`phase1/step2-ground-truth-evaluator.ts`), which analysed all 389 labelled items in the dataset. Table 4.3.1 summarises these metrics and highlights the clear performance separation between models:

Model	Score	Precision	Recall	F1	Overall Accuracy	Balanced Accuracy
Claude 3.7 Sonnet	351 / 389 correct	90.70%	90.20%	90.50%	90.20%	77.30%
Claude 4 Sonnet	330 / 389 correct	85.30%	84.80%	85.10%	84.80%	74.80%
Mixtral 8x7B	270 / 389 correct	69.80%	69.40%	69.60%	69.40%	60.80%
Mistral 7B	232 / 389 correct	60.40%	59.60%	60.00%	59.60%	49.20%

Table 4.3.1 – Model Accuracy Metrics Summary

Table 4.3.1 shows a clear performance separation. Claude 3.7 leads all models, with Claude 4 close behind, while Mixtral and Mistral lag by a wide margin. The results indicate that the Claude family produces more consistent, correctly structured outputs, which align with the lower hallucination rates observed during evaluation. For production use, these findings suggest that Claude 3.7 offers the best balance between precision and recall, while open-weight models trade accuracy for speed and cost, a trade-off unlikely to be acceptable for ReceiptCAT.

Per-Category Accuracy Comparisons

Per-category accuracy was calculated by comparing each model’s predicted label for every item against the corresponding `ground_truth_category` value in the manually verified dataset. These calculations were performed automatically by the evaluator script (`phase1/step2-ground-truth-evaluator.ts`), which grouped predictions by category and computed the proportion of correct classifications within each group. This approach highlights differences in model performance across both common and infrequent expense types while reducing the influence of dominant categories.

Table 4.3.2 presents the per-category accuracy results for all four models. Categories with more examples (such as “*Pantry & Snacks*” or “*Eating Out*”) provide a stable basis for comparison, while those with very few samples can vary sharply because even a single error can substantially alter the percentage.

Category Identifier	Count	Claude 3.7	Claude 4	Mixtral 8x7B	Mistral 7B
fresh_produce	31	100.00%	100.00%	90.30%	74.20%
meat_seafood	22	86.40%	72.70%	77.30%	63.60%
dairy_eggs	30	86.70%	86.70%	80.00%	70.00%
pantry_snacks	90	96.70%	87.80%	67.80%	56.70%
bakery	17	88.20%	88.20%	76.50%	70.60%
coffee_tea	14	85.70%	100.00%	64.30%	71.40%
beverages	25	80.00%	76.00%	76.00%	72.00%
alcohol	5	80.00%	0.00%	40.00%	40.00%
eating_out	61	98.40%	100.00%	82.00%	49.20%
health_medicine	6	100.00%	100.00%	100.00%	100.00%
personal_care_beauty	28	100.00%	92.90%	21.40%	64.30%
home_cleaning	21	85.70%	61.90%	42.90%	33.30%
baby_maternity	0	[N/A]	[N/A]	[N/A]	[N/A]
pets	1	0.00%	100.00%	0.00%	0.00%
clothing_footwear	10	90.00%	90.00%	80.00%	90.00%
electronics_tech	1	100.00%	100.00%	100.00%	100.00%
stationery_office	4	50.00%	75.00%	100.00%	25.00%
sports_fitness	1	100.00%	100.00%	0.00%	0.00%
gifts_occasions	2	100.00%	100.00%	0.00%	0.00%
entertainment	0	[N/A]	[N/A]	[N/A]	[N/A]
subscriptions_digital	1	100.00%	100.00%	100.00%	0.00%
professional_services	6	33.30%	16.70%	83.30%	33.30%
utilities_bills	5	60.00%	0.00%	60.00%	60.00%
transport_fuel	0	[N/A]	[N/A]	[N/A]	[N/A]
travel_holidays	1	0.00%	0.00%	0.00%	0.00%
other	7	57.10%	71.40%	57.10%	57.10%

Table 4.3.2 – Per-Category Accuracy by Model

Categories with larger sample sizes remain strong for the Claude models, particularly “*Eating Out*”, “*Fresh Produce*”, and “*Pantry & Snacks*”. Performance gaps widen in household and personal-care categories where item descriptions are less standardised, with the open-weight models showing more frequent misclassifications

or apparent category-level hallucinations. These errors typically occur between semantically adjacent categories rather than as random outputs, reflecting the natural ambiguity of many receipts.

Results for very small categories vary widely because single predictions can disproportionately affect percentages. “*Pets*”, “*Gifts & Occasions*”, and “*Subscriptions & Digital Services*” illustrate this sensitivity and should therefore be interpreted with caution. Overall, the pattern reinforces the need for balanced-accuracy reporting and per-category analysis to offset category imbalance. Using these measures ensures each category contributes equally to the overall performance score, preventing high-volume classes from dominating the evaluation and masking weaknesses in low-frequency categories. This makes it easier to isolate which specific categories require targeted improvements, such as refined prompts or additional training examples. In practice, Claude 3.7 demonstrates the highest stability across common categories, suggesting that targeted disambiguation rules or modest data augmentation could further improve performance in sparse classes during later phases.

Output Array-length Accuracy Comparisons

Array-length consistency was evaluated by comparing the number of elements in each model’s JSON output array with the number of detected items in the ground-truth data for every receipt. A mismatch occurred when these two counts differed, indicating that the model either produced extra category labels, omitted some items, or returned a truncated array. The mismatch rate is the proportion of receipts with at least one such discrepancy among the 118 receipts processed.

Model	Mismatches	Rate
Claude 3.7	0 / 118	0.00%
Claude 4	0 / 118	0.00%
Mixtral 8x7B	2 / 118	~1.9%
Mistral 7B	10 / 118	~9.3%

Table 4.3.3 – Output Array Consistency vs Ground Truth

Array-length consistency was perfect for the Claude models, which always returned a JSON array matching the ground-truth item count, eliminating downstream parsing errors and category-level hallucinations. By contrast, both open-weight models produced occasional to frequent mismatches, with Mixtral showing minor drift and Mistral exhibiting the highest instability, including extra labels on single-item receipts. These discrepancies indicate weaker adherence to schema constraints and increase the risk of hallucinated or truncated outputs in production.

Runtime Performance Comparisons

The runtime performance results show that the open-weight models are generally faster, with Mixtral 8x7B leading in mean latency, while Claude 3.7 is consistently quicker than Claude 4 within the Claude family. However, tail behaviour matters: Mistral 7B exhibits extreme worst-case latency, and both open-weight models show higher variance that coincides with hallucinated or malformed outputs. In contrast, the Claude models are slower but steadier, trading a few hundred milliseconds for more reliable, schema-conformant responses.

Model	Mean (ms)	Min (ms)	Max (ms)
Claude 3.7	1265	873	3078
Claude 4	1433	938	2852
Mixtral 8x7B	431	208	2144
Mistral 7B	912	172	15351

Table 4.3.4 – Runtime Performance (ms)

Token Usage Comparisons

Token accounting was available only for the Claude models, which consistently reported both input and output token counts. On average, each receipt classification required roughly 420 input tokens and 24 output tokens, resulting in an estimated cost of \$0.004 per call under Bedrock’s published pricing [5]. This equates to approximately 250 inferences per dollar. Token usage between Claude 3.7 and Claude 4 was nearly identical, confirming that prompt and output lengths were well controlled across both models. Open-weight models such as Mixtral and Mistral did not provide token metrics, but given their smaller architecture and shorter prompts, their operational costs are expected to be lower, though this could not be quantified within Bedrock’s reporting framework.

Model	Avg Input Tokens	Min Input	Max Input	Avg Output Tokens	Min Output	Max Output
Claude 3.7 Sonnet	421	336	1083	24	6	165
Claude 4 Sonnet	422	337	1084	24	6	156

Table 4.3.5 – Token Usage (Claude Models Only)

Model	Avg Input Cost/Call	Avg Output Cost/Call	Avg Total Cost/Call	Calls per \$1
Claude 3.7 Sonnet	\$0.003370	\$0.000580	\$0.003950	250
Claude 4 Sonnet	\$0.003380	\$0.000580	\$0.003960	250

Table 4.3.6 – Estimated Cost per Call (Claude Models Only)

3.4 PHASE 1 CONCLUSIONS

The **Claude 3.7 Sonnet** foundational model offers the best balance for ReceiptCAT, combining high accuracy with steady runtimes and perfectly aligned output lengths. Claude 4 Sonnet performs well but is slower on average, so it does not improve the overall trade-off. The open-weight models are generally faster and likely cheaper to operate, yet they were markedly less reliable, with frequent array-length mismatches and category-level hallucinations. On this basis, Claude 3.7 is the recommended Phase 1 winner to carry into Phase 2 for targeted refinements.

4 PHASE 2 EVALUATION

4.1 OBJECTIVE

The objectives in this phase are to improve and validate the Phase 1 winner (Claude 3.7 Sonnet) by testing prompt-level refinements and selecting a production configuration with the best accuracy-to-cost balance. We compare variants that update the predefined categories, introduce rule-based disambiguation, and add a few-shot examples. Each variant is evaluated on the same dataset using the Phase 1 methodology, with results reported for overall and balanced accuracy, precision, recall, F1 score, runtime, and estimated cost per call based on token usage. We also run a lenient check that accepts plausible alternative categories to quantify inherent label ambiguity. The outcome of Phase 2 is a single recommended configuration and clear guidance on trade-offs for production.

4.2 METHODOLOGY

Evaluation design

Phase 2 followed the same evaluation process used in Phase 1, focusing exclusively on prompt-level refinements to the Claude 3.7 Sonnet model. The primary goal was to determine whether structured prompt updates could

improve classification accuracy without compromising cost efficiency or response consistency. Unless otherwise stated, all configuration parameters, dataset inputs, and evaluation metrics remained identical to Phase 1 to ensure comparability. The subsections below describe the elements unique to Phase 2, including updates to the predefined category set, prompt variants, evaluation procedure, data analysis, and cost considerations.

Phase 2 followed the same evaluation process used in Phase 1, focusing exclusively on prompt-level refinements to the Claude 3.7 Sonnet model. The primary goal was to determine whether structured prompt updates could improve classification accuracy without compromising cost efficiency or response consistency. Unless otherwise stated, all configuration parameters, dataset inputs, and evaluation metrics remained identical to Phase 1 to ensure comparability.

The evaluation process was implemented using custom TypeScript scripts in the **bedrock-evaluation-scripts** subfolder of the **P000340SE-EvaluationReport-Appendix.zip** archive. These include:

- **phase2/step1-model-evaluator.ts** – Executes all prompt variants sequentially, invoking the Claude 3.7 Sonnet model through the AWS Bedrock InvokeModel API and logging outputs, tokens, and latency data.
- **phase2/step2-ground-truth-evaluator.ts** – Computes accuracy and performance metrics using the labelled file `bedrock_phase2_ground_truth_eval.csv`, which contains the manually verified `ground_truth_category_p2` and optional `ground_truth_category_p2_alt` columns for strict and lenient evaluation.
- **phase2/prompt.ts** – Defines all Phase 2 prompt variants, including schema definitions, disambiguation rules, and few-shot examples.

The same cost-control measures from Phase 1 were retained: all experiments were executed locally using only the Bedrock InvokeModel API, with results cached to avoid redundant inferences. The main cost difference in Phase 2 arose from longer prompt text and, in the few-shot variant, embedded examples, both of which increased input token counts. Output length remained stable across variants. As a result, input token usage became the primary cost driver, with the most tuned variant roughly doubling the per-call cost compared to the baseline. These findings were integrated into the accuracy-to-cost comparison to support model selection for deployment.

Prompt Variants and Design Adjustments

Phase 2 tested four prompt configurations derived from the Claude 3.7 Sonnet baseline (renamed for clarity as `claude37_p1`, `claude37_p2`, etc.), each applying a progressively richer set of instructions:

1. **Baseline (`claude37_p1`):** Original Phase 1 baseline, using the original predefined category list (see Table 4.2.1), and no rule-based adjustments or examples.
2. **Updated Categories (`claude37_p2`):** Revised category list (see Table 5.2.1 below) with the same schema, but no additional heuristics or examples.
3. **Updated Categories + Rules (`claude37_p2_rules`):** Added explicit classification rules and disambiguation heuristics to minimise overlap (e.g., “If an item is refrigerated or perishable, classify it under *dairy_eggs_fridge* rather than *pantry_snacks*”).
4. **Updated Categories + Rules + Few-Shot (`claude37_p2_rules_fs`):** Retained the rules from Variant 3 and appended several worked examples to illustrate the expected format and logic.

Each prompt retained the schema-first design from Phase 1, requiring output as a JSON array whose length matched the number of detected items. Wording was kept concise to balance clarity and token efficiency. The classification rules covered the full category set and included boundaries such as:

- Distinguishing refrigerated or perishable items from shelf-stable goods (*dairy_eggs_fridge* vs *pantry_snacks*).

- Separating drinkable liquids from pantry items (*drinks vs coffee_tea*).
- Prioritising context clues such as vendor type or packaging keywords (*café or takeaway → eating_out*).
- Handling ambiguous personal or household items (*personal_care_beauty vs cleaning_maintenance*).

These rules were formulated to guide consistent boundary handling rather than promote memorisation, following AWS Bedrock’s official prompt engineering recommendations [4]. The complete rule set, along with all schema definitions and few-shot examples, is available in the `phase2/prompt.ts` file within the `P000340SE-EvaluationReport-Appendix.zip` archive.

Updated Predefined Category Set

The baseline category list from Phase 1 was revised following a review of ambiguous or overlapping labels observed during data annotation and model evaluation. Several receipt items, such as frozen dumplings, hommus, deli meats, and iced coffee drinks, highlighted weaknesses in the earlier set, particularly in distinguishing between perishable goods and cold-storage items. To address this, the updated list introduced finer-grained categories that clarified boundaries between similar domains (for example, defining “*Dairy & Eggs & Fridge*” to separate fridge items from drinks and pantry items).

#	Category Name	Identifier
1	Fruits & Vegetables	fruits_vegetables
2	Meat & Seafood & Deli	meat_seafood_deli
3	Dairy & Eggs & Fridge	dairy_eggs_fridge
4	Frozen	frozen
5	Pantry & Snacks	pantry_snacks
6	Bakery	bakery
7	Coffee & Tea	coffee_tea
8	Drinks	drinks
9	Liquor	liquor
10	Eating Out	eating_out
11	Health & Medicine	health_medicine
12	Personal Care & Beauty	personal_care_beauty
13	Cleaning & Maintenance	cleaning_maintenance
14	Baby & Maternity	baby_maternity
15	Pets	pets
16	Clothing & Footwear	clothing_footwear
17	Electronics & Tech	electronics_tech
18	Home & Lifestyle	home_lifestyle
19	Sports & Fitness	sports_fitness
20	Gifts & Occasions	gifts_occasions
21	Entertainment	entertainment
22	Subscriptions & Digital Services	subscriptions_digital
23	Professional Services	professional_services
24	Utilities & Bills	utilities_bills
25	Transport & Fuel	transport_fuel
26	Travel & Holidays	travel_holidays
27	Other	other

Table 5.2.1 – Updated Predefined Category Set used in Phase 2’s Model Evaluation

These refinements were developed collaboratively within the development team based on discussions with the Product Owner, resulting in a more practical and representative mapping of real-world purchase data. The revised list, shown in Table 5.2.1, was applied consistently across all Phase 2 prompts and evaluations.

Each prompt retained the schema-first design from Phase 1, requiring output as a JSON array whose length exactly matched the number of detected items. Wording was kept concise to balance clarity and token efficiency. Rules and few-shot examples were formulated to guide the model toward consistent boundary handling rather than memorising examples, in line with AWS Bedrock's prompt engineering recommendations [4].

Evaluation Procedure

The same dataset of 118 receipts (389 expense items) and orchestration process from Phase 1 were reused to ensure consistency. All four prompt variants were tested using identical invocation settings via AWS Bedrock's `InvokeModel` API.

Ground-truth categories were manually verified and updated by two members of the ReceiptCAT development team (Gabby Sanchez and Sneha Thankam Oommen) prior to evaluation. These updates were integrated into the file `bedrock_phase2_ground_truth_eval.csv`, which contains both `ground_truth_category_p2` and `ground_truth_category_p2_alt` columns. Once this file was prepared, all evaluation metrics were computed automatically by the evaluator script (`phase2/step2-ground-truth-evaluator.ts`), ensuring consistency and reproducibility across all prompt variants.

Evaluation continued under two modes:

- **Strict evaluation**, identical to Phase 1, where a prediction was considered correct only if it exactly matched the ground-truth category.
- **Lenient evaluation**, newly introduced in Phase 2, where a prediction was counted as correct if it matched either the primary category or a valid alternative category, accounting for naturally ambiguous receipt items (e.g., *iced coffee* could reasonably belong to either *drinks* or *dairy_eggs_fridge*).

This dual-mode approach provided a more nuanced view of classification accuracy, distinguishing between clear misclassifications and contextually acceptable alternatives.

Data Analysis

Each model variant was analysed using the same metrics and tools as in Phase 1, including overall and balanced accuracy, precision, recall, F1 score, and per-category accuracy tables. In addition to accuracy metrics, each model variant was also compared on runtime, token counts, and estimated per-call costs to assess the trade-off between improved accuracy and computational efficiency. Phase 2 analysis also quantified the proportion of inherently ambiguous items (approximately 20 of the 389 total) that affected strict accuracy but were acceptable under lenient evaluation.

Manual verification was conducted for a sample of receipts to confirm that improvements were attributable to prompt refinements rather than incidental variation. Comparative tables and graphs summarised these results across all four variants to identify the most balanced configuration for production.

Cost-Optimisation Considerations

The same cost-control measures from Phase 1 were retained: all experiments were executed locally using only the Bedrock `InvokeModel` API, with results cached to avoid redundant inferences. The main cost difference in Phase 2 arose from longer prompt text and, in the few-shot variant, embedded examples, both of which increased input token counts. Output length remained stable across variants. As a result, input token usage became the

primary cost driver, with the most tuned variant roughly doubling the per-call cost compared to the baseline. These findings were integrated into the accuracy-to-cost comparison to support model selection for deployment.

4.3 FINDINGS

Strict Evaluation - Overall Accuracy Comparisons

The strict evaluation shows clear gains from prompt refinements. Updating the categories alone yields a small lift in overall metrics and a larger improvement in balanced scores, indicating better handling of minority classes. Adding rules/heuristics delivers the strongest results, with overall precision/recall/F1 around 93% and balanced metrics also improving over the baseline, suggesting that explicit disambiguation reduces overlap errors across adjacent categories.

Model Variant	Metric	Precision	Recall	F1
Claude 3.7 Baseline	Overall Accuracy	90.70%	90.20%	90.50%
Claude 3.7 Baseline	Balanced Accuracy	81.40%	77.30%	77.20%
Claude 3.7 Updated Categories (No-shot)	Overall Accuracy	90.70%	90.50%	90.60%
Claude 3.7 Updated Categories (No-shot)	Balanced Accuracy	77.20%	82.20%	77.40%
Claude 3.7 Updated Categories + Rules (No-shot)	Overall Accuracy	93.30%	93.10%	93.20%
Claude 3.7 Updated Categories + Rules (No-shot)	Balanced Accuracy	80.80%	83.90%	80.80%
Claude 3.7 Updated Categories + Rules + Few-shot	Overall Accuracy	92.50%	92.30%	92.40%
Claude 3.7 Updated Categories + Rules + Few-shot	Balanced Accuracy	80.20%	83.20%	79.80%

Table 5.3.1 – Strict Evaluation: Micro (Overall) and Balanced Metrics

By contrast, few-shot examples do not outperform the rules-only variant under strict scoring and add input tokens. This aligns with qualitative observations: many of the ~20/389 inherently ambiguous items (e.g., *iced coffee* as *drinks* vs *dairy_eggs_fridge*) penalise strict metrics regardless of examples. Overall, the Updated Categories + Rules (No-shot) model variant achieves the best accuracy while avoiding unnecessary token growth from examples.

Lenient Evaluation - Overall Accuracy Comparisons

Under a lenient criterion (counting a prediction as correct if it matches either the primary label or a valid alternative), accuracy improves across all tuned variants. The **Updated Categories + Rules (No-shot)** model variant ranks highest, achieving ~95% overall and 85.8% balanced accuracy, while the **Few-shot** variant is essentially tied on overall accuracy but not materially better on balanced accuracy. The lift is concentrated in previously ambiguous areas; for example, items such as *iced coffee* or *hommus* no longer penalise the model when mapped to a defensible alternative. In practice, this means only about 5% (20/389) of items are truly incorrect under realistic user expectations.

Model Variant	Metric	Precision	Recall	F1
Claude 3.7 Updated Categories (No-shot)	Overall Accuracy	91.80%	91.50%	91.60%
Claude 3.7 Updated Categories (No-shot)	Balanced Accuracy	81.20%	82.90%	80.80%
Claude 3.7 Updated Categories + Rules (No-shot)	Overall Accuracy	95.60%	95.40%	95.50%
Claude 3.7 Updated Categories + Rules (No-shot)	Balanced Accuracy	86.30%	85.80%	85.40%
Claude 3.7 Updated Categories + Rules + Few-shot	Overall Accuracy	95.40%	95.10%	95.20%
Claude 3.7 Updated Categories + Rules + Few-shot	Balanced Accuracy	90.40%	89.90%	89.60%

Table 5.3.2 – Lenient Evaluation: Micro (Overall) and Balanced Metrics

Per-category patterns are consistent with Phase 1: strong stability in high-support classes (e.g., *fruits_vegetables*, *frozen*, *eating_out*), large gains for *dairy_eggs_fridge* under rules, and continued weakness for drinks, where boundary definitions remain fuzzy. Small-sample categories remain volatile and should be interpreted cautiously.

Category	Count	claude37_p2	claude37_p2_rules	claude37_p2_rules_fs
fruits_vegetables	26	100.00%	100.00%	100.00%
meat_seafood_deli	17	100.00%	88.20%	94.10%
dairy_eggs_fridge	41	78.00%	100.00%	100.00%
frozen	19	100.00%	100.00%	100.00%
pantry_snacks	73	98.60%	98.60%	95.90%
bakery	15	86.70%	86.70%	93.30%
coffee_tea	14	100.00%	100.00%	100.00%
drinks	23	69.60%	56.50%	56.50%
liquor	5	80.00%	80.00%	80.00%
eating_out	61	96.70%	100.00%	98.40%
health_medicine	11	90.90%	90.90%	81.80%
personal_care_beauty	28	100.00%	89.30%	92.90%
cleaning_maintenance	11	100.00%	90.90%	90.90%
baby_maternity	0	[N/A]	[N/A]	[N/A]
pets	1	0.00%	100.00%	100.00%
clothing_footwear	10	80.00%	80.00%	80.00%
electronics_tech	1	100.00%	100.00%	100.00%
home_lifestyle	15	93.30%	93.30%	86.70%
sports_fitness	1	100.00%	100.00%	100.00%
gifts_occasions	1	100.00%	100.00%	0.00%
entertainment	0	[N/A]	[N/A]	[N/A]
subscriptions_digital	1	100.00%	100.00%	100.00%
professional_services	1	100.00%	100.00%	100.00%
utilities_bills	5	60.00%	60.00%	60.00%
transport_fuel	0	[N/A]	[N/A]	[N/A]
travel_holidays	1	0.00%	0.00%	0.00%
other	8	50.00%	87.50%	75.00%

Table 5.3.3 – Lenient Evaluation: Per-Category Accuracy by Variant

Runtime Performance Comparisons

The baseline variant is marginally faster on average than the tuned prompts, with small step-ups in mean latency as rules and few-shot examples are added. The differences remain modest, indicating that richer instructions do not materially affect runtime, though they slightly increase tail latency (the highest max for the few-shot variant). This pattern is consistent with longer prompt text driving small overheads, while evaluation mode (strict vs lenient) does not affect inference time.

Model Variant	Mean (ms)	Min (ms)	Max (ms)
Claude 3.7 Baseline	1265	873	3078
Claude 3.7 Updated Categories (No-shot)	1359	861	3367
Claude 3.7 Updated Categories + Rules (No-shot)	1398	985	3364
Claude 3.7 Updated Categories + Rules + Few-shot	1439	972	3739

Table 5.3.4 – Lenient Evaluation: Runtime Performance (ms)

Token Usage Comparisons

Input tokens increased with tuning, mainly due to the added rules and few-shot examples, while output tokens remained nearly constant across all variants. As a result, the estimated cost per call rose with each step of prompt refinement, with the rules-only variant roughly doubling the baseline cost and the rules plus few-shot variant adding the most overhead. This reflects a clear trade-off: improvements in accuracy from structured prompts come at the cost of higher token usage, while output length and cost remain stable.

Model Variant	Avg Input Tokens	Min Input	Max Input	Avg Output Tokens	Min Output	Max Output
Claude 3.7 Baseline	421	336	1083	24	6	165
Claude 3.7 Updated Categories (No-shot)	433	348	1095	26	6	177
Claude 3.7 Updated Categories + Rules (No-shot)	979	894	1641	26.1	6	176
Claude 3.7 Updated Categories + Rules + Few-shot	1212	1127	1874	26.5	6	176

Table 5.3.5 –Token Usage (Claude 3.7 Variants)

Model Variant	Avg Input Cost/Call	Avg Output Cost/Call	Avg Total Cost/Call	Calls per \$1
Claude 3.7 Baseline	\$0.003370	\$0.000580	\$0.003950	253
Claude 3.7 Updated Categories (No-shot)	\$0.003460	\$0.000620	\$0.004160	240
Claude 3.7 Updated Categories + Rules (No-shot)	\$0.007830	\$0.000630	\$0.008740	114
Claude 3.7 Updated Categories + Rules + Few-shot	\$0.009700	\$0.000640	\$0.010670	94

Table 5.3.6 – Estimated Cost per Call (Claude 3.7 Variants)

4.4 PHASE 2 CONCLUSIONS

The **Claude 3.7 Sonnet** foundational model, **coupled with updated categories and rules (no-shot)**, achieved the highest strict and lenient accuracies, confirming that clear category definitions and simple disambiguation rules reduce overlap errors. The trade-off is cost: this variant roughly doubles per-call spend compared to the baseline due to longer prompts. If the budget is tighter, **updated categories only (no-shot)** offers a strong middle ground, delivering meaningful accuracy gains with modest cost and runtime increases. Few-shot prompting increased the number of input tokens and cost without improving results beyond the rules-only variant, so it is not recommended for production.

5 OVERALL CONCLUSIONS

Across both evaluation phases, **Claude 3.7 Sonnet** consistently demonstrated the best balance of accuracy, consistency, and cost efficiency for ReceiptCAT's receipt-item classification task. In Phase 1, Claude 3.7 established a strong baseline, achieving high overall accuracy and zero array-length mismatches. The open-weight models Mixtral 8x7B and Mistral 7B performed faster but were less reliable, frequently producing inconsistent or hallucinated outputs. Phase 2 confirmed that refining the predefined categories and introducing rule-based heuristics further improved accuracy, while few-shot prompting added cost without meaningful gains.

After reviewing both phases, the client and development team agreed that **Claude 3.7 Sonnet, with the updated predefined categories and no-shot prompting**, provides the optimal trade-off between accuracy and cost. This configuration consistently achieved high performance across both strict and lenient evaluations while keeping runtime and token usage within reasonable limits. It has therefore been selected as the receipt categoriser model for ReceiptCAT, representing the best balance between practical accuracy and long-term cost optimisation.

6 FURTHER WORK

Future work will extend this evaluation by fine-tuning the best-performing model: Claude 3.7 Sonnet with updated predefined categories and no-shot prompting. Further experiments could explore lightweight model pre-training or targeted fine-tuning to improve category boundary recognition without substantially increasing token usage or cost. This would help reduce dependence on prompt rules while maintaining or improving classification accuracy.

Another priority is expanding and rebalancing the dataset. Some categories remain underrepresented or are missing entirely, limiting the reliability of balanced accuracy metrics. Collecting additional receipts, particularly for categories such as *"Baby and Maternity"*, *"Entertainment"*, and *"Transport & Fuel"*, would enable a more robust evaluation across the full taxonomy and strengthen the model's performance consistency in future phases.

REFERENCES

1. Amazon Web Services, “Amazon Bedrock Developer Guide,” *AWS Documentation*, 2025. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide>
2. Amazon Web Services, “InvokeModel API Reference,” *Amazon Bedrock API Reference*, 2025. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_InvokeModel.html
3. Amazon Web Services, “Supported foundation models in Amazon Bedrock,” *AWS Documentation*, 2025. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/models-supported.html>
4. Amazon Web Services, “Prompt Engineering in Amazon Bedrock,” *AWS Documentation*, 2025. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-engineering-guidelines.html>
5. Amazon Web Services, “Pricing for Amazon Bedrock,” *AWS Pricing Documentation*, 2025. [Online]. Available: <https://aws.amazon.com/bedrock/pricing/>